Module 1 – Algorithm Analysis and Basic Concepts

Dr. Vignesh Ramamoorthy. H

Associate Professor

AIIT – AUB

Module 1 – Algorithm Analysis and Basic Concepts

- Introduction to Data Structures and Algorithms
- Abstract Data Type (ADT)
- Need for efficient algorithms
- Time Complexity and Space Complexity
- Asymptotic Notations: Big-O, Big-Ω, Big-Θ
- Best, Average and Worst case analysis
- Growth rate of functions (constant, logarithmic, linear, quadratic)

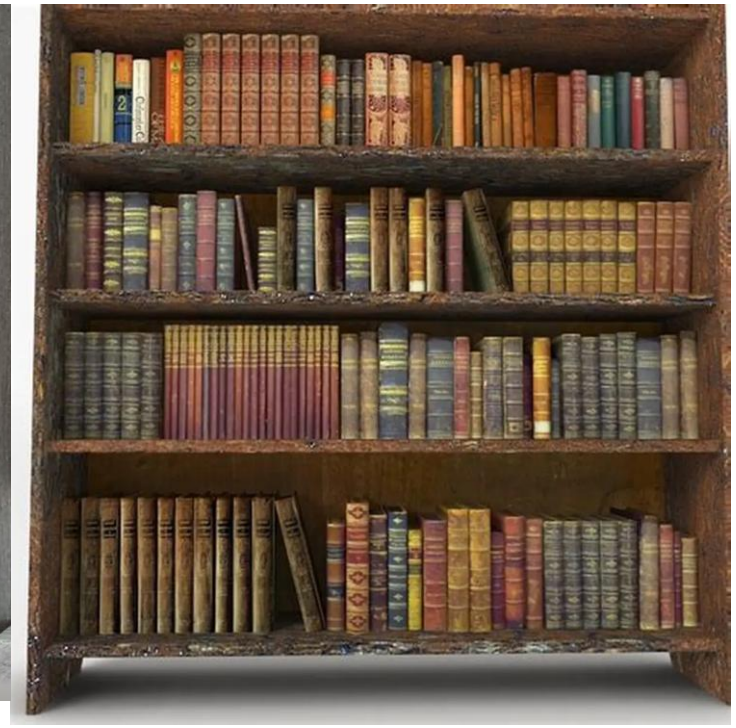
Module 1 – Algorithm Analysis and Basic Concepts

- **Introduction to Data Structures and Algorithms**
- Abstract Data Type (ADT)
- Need for efficient algorithms
- Time Complexity and Space Complexity
- Asymptotic Notations: Big-O, Big-Ω, Big-Θ
- Best, Average and Worst case analysis
- Growth rate of functions (constant, logarithmic, linear, quadratic)

Introduction to Data Structures and Algorithms

Introduction to Data Structures and Algorithms

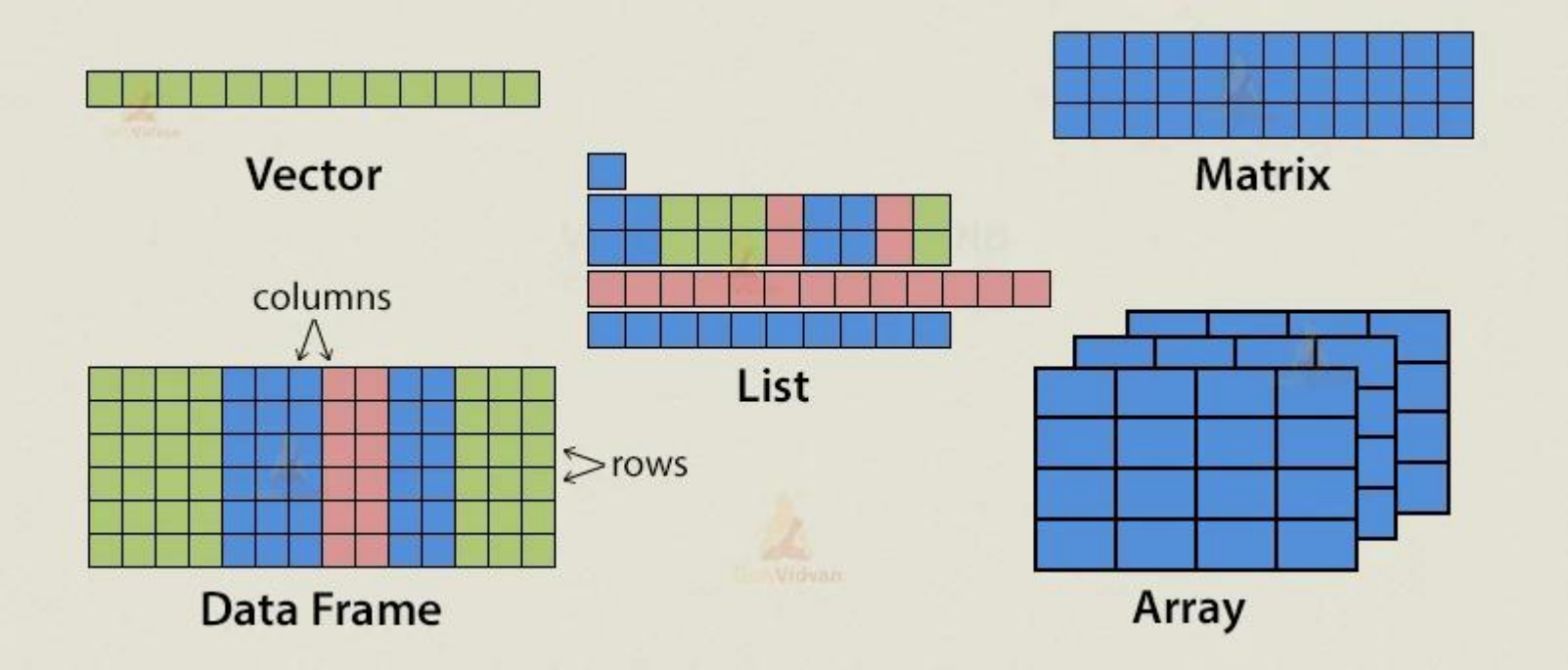
A **Data Structure** is a way of organizing, managing, and storing data so that it can be used efficiently.



Introduction to Data Structures and Algorithms

Example - Cupboard:

- The cupboard = data structure
- Clothes/books = data (information stored)
- Different cupboards organize things differently (shelves, drawers, hangers).



Introduction to Data Structures and Algorithms

Data structures organize data in a program and represent different **datatypes** in **physical form**.

Every data structure defines:

- The **method of connecting** a group of elements and their **in-memory representation**.
- The allowed set of **operations** and **algorithms** over the grouped elements.

All operations fall into one of the two following categories:

- **Queries**, which return information about a data structure without making changes.
- **Modifiers**, which change the contents of a data structure.

Introduction to Data Structures and Algorithms

AMITY UNIVERSITY BENGALURU

What are Data Structures Used for?

- Data structures provide an efficient way to store and access large amounts of data.
- Various fields of applied programming, such as **AI**, **databases**, and others, research the problem of efficient data storage.
- Data structures, together with **algorithms**, are the core of any program.
- Well-organized data structures help design structured algorithms and **reduce time and memory complexity**.

Introduction to Data Structures and Algorithms

AMITY UNIVERSITY BENGALURU

Types of Data Structures

1. Primitive Data Structures

- Basic types built into programming languages
- Examples: Integer, Float, Character, Boolean.

2. Non-Primitive Data Structures

- More complex, built using primitives.
- Examples:
  - ✓ **Linear Data Structures** → Arrays, Linked Lists, Stacks, Queues
  - ✓ **Non-Linear Data Structures** → Trees, Graphs

What is an Algorithm?

An **Algorithm** is a step-by-step procedure to solve a problem or perform a computation.

Searching for a Word in a Dictionary

- **Problem:** ***Find the word "Tiger".***
- **Algorithm:**
  - ✓ **Open the dictionary in the middle.**
  - ✓ **Check if the word is before or after the middle section.**
  - ✓ **Narrow down to the correct section.**
  - ✓ **Continue until you find "Tiger".**
- **This is like Binary Search Algorithm.**

In programming, algorithms define **clear instructions** to solve computational problems.

AMITY UNIVERSITY BENGALURU

Properties of Algorithms

- **Input**: Zero or more inputs
- **Output**: At least one result
- **Finiteness**: Must terminate after a finite number of steps
- **Definiteness**: Each step must be clear
- **Effectiveness**: Must be feasible to execute

Module 1 – Algorithm Analysis and Basic Concepts

- Introduction to Data Structures and Algorithms
- **Abstract Data Type (ADT)**
- Need for efficient algorithms
- Time Complexity and Space Complexity
- Asymptotic Notations: Big-O, Big-Ω, Big-Θ
- Best, Average and Worst case analysis
- Growth rate of functions (constant, logarithmic, linear, quadratic)

Abstract Data Type (ADT)

- An **Abstract Data Type (ADT)** is a conceptual model that defines a set of operations and behaviors for a data structure, **without specifying how these operations are implemented** or how data is organized in memory.
- The definition of ADT only mentions what **operations are to be performed** but not **how** these operations will be implemented.
- It does not specify how data will be organized in memory and what algorithms will be used for implementing the operations.
- **It is called "abstract" because it provides an implementation-independent view.**
- The process of providing only the essentials and hiding the details is known as abstraction.

Abstract Data Type (ADT)

An Abstract Data Type (ADT) is like a blueprint or idea of a data structure

- It defines the **behavior or operations** (what you can do).
- It hides the **implementation details** (how it's done).

Abstract Data Type (ADT)

Features of ADT

Abstract data types (ADTs) are a way of encapsulating data and operations on that data into a single unit. Some of the key features of ADTs include:

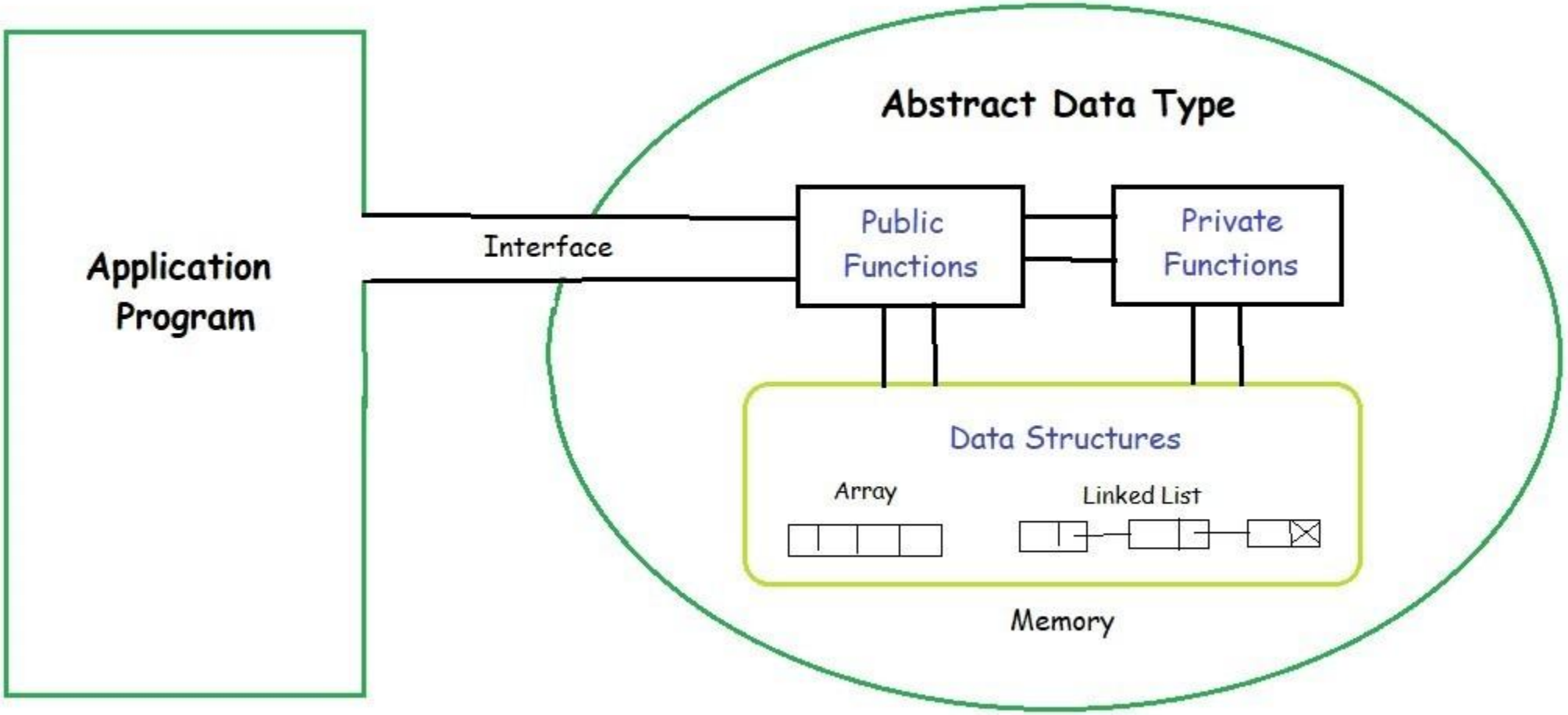
- **Abstraction:** The user does not need to know the implementation of the data structure only essentials are provided.
- **Better Conceptualization:** ADT gives us a better conceptualization of the real world.
- **Robust:** The program is robust and has the ability to catch errors.
- **Encapsulation**: ADTs hide the internal details of the data and provide a public interface for users to interact with the data. This allows for easier maintenance and modification of the data structure.

Abstract Data Type (ADT)

Features of ADT

- **Data Abstraction**: ADTs provide a level of abstraction from the implementation details of the data. Users only need to know the operations that can be performed on the data, not how those operations are implemented.
- **Data Structure Independence**: ADTs can be implemented using different data structures, such as arrays or linked lists, without affecting the functionality of the ADT.
- **Information Hiding:** ADTs can protect the integrity of the data by allowing access only to authorized users and operations. This helps prevent errors and misuse of the data.
- **Modularity**: ADTs can be combined with other ADTs to form larger, more complex data structures. This allows for greater flexibility and modularity in programming.

Abstract Data Type (ADT)



Abstract Data Type (ADT)

AMITY UNIVERSITY BENGALURU

Why Use ADTs?

The key reasons to use ADTs in Java are listed below:

- **Encapsulation:** Hides complex implementation details behind a **clean interface.**
- **Reusability**: Allows different internal implementations (e.g., array or linked list) without changing external usage.
- **Modularity:** Simplifies maintenance and updates by separating logic.
- **Security:** Protects data by preventing direct access, minimizing bugs and unintended changes.

Abstract Data Type (ADT)

Relation Between Data Structures, Algorithms, and ADT

- **ADT** → defines *what* operations are needed.
- **Data Structure** → provides a concrete way to *implement* the ADT.
- **Algorithm** → provides the *logic/steps* to perform operations efficiently.

Example:

- **ADT**: Stack (operations = push, pop, peek)
- **Data Structure**: Can be implemented using an **array** or a **linked list**
- **Algorithm**: Steps to perform push/pop efficiently

Comparison of ADT and DS

Aspect	**ADT** (Abstract Data Type)	**Data Structure** (Concrete Implementation)
What it is	Logical description of operations	Actual way of storing data in memory
Focus	What can be done (operations)	How it is done (implementation)
Example	Stack (push, pop)	Stack using array / linked list
Analogy	ATM functions	Circuits, software, and hardware inside ATM

Module 1 – Algorithm Analysis and Basic Concepts

- Introduction to Data Structures and Algorithms
- Abstract Data Type (ADT)
- **Need for efficient algorithms**
- Time Complexity and Space Complexity
- Asymptotic Notations: Big-O, Big-Ω, Big-Θ
- Best, Average and Worst case analysis
- Growth rate of functions (constant, logarithmic, linear, quadratic)

Need for Efficient Algorithms

What is an Algorithm?

An algorithm is a **step-by-step procedure** to solve a problem.

In a **Data Structures**, algorithms are used to **process data.**

(searching, sorting, inserting, deleting, etc.).

If an algorithm is **inefficient**, it may:

- Take too much time
- Use too much memory
- Become unusable for large data

Need for Efficient Algorithms

Example

Imagine searching for a name in:

- 10 students → Easy
- 10,00,000 students → Slow if method is poor

If you check one by one → Slow

If data is sorted and you use Binary Search → Fast

Efficient algorithms save **time and memory**.

Need for Efficient Algorithms

Goals of Algorithm Analysis

1) Measure performance
2) Compare different algorithms
3) Predict scalability
4) Choose best solution

Module 1 – Algorithm Analysis and Basic Concepts

- Introduction to Data Structures and Algorithms
- Abstract Data Type (ADT)
- Need for efficient algorithms
- **Time Complexity and Space Complexity**
- Asymptotic Notations: Big-O, Big-Ω, Big-Θ
- Best, Average and Worst case analysis
- Growth rate of functions (constant, logarithmic, linear, quadratic)

Time Complexity

- **Time Complexity is the measure of the amount of time an algorithm takes to complete as a function of the input size (n).**
- It does **not** measure actual time in seconds.
- It measures **how the number of operations grows** when input size increases.
- Time Complexity tells us:

  "If input becomes bigger, how much slower does the algorithm become?"

Time Complexity

How to Calculate Time Complexity?

Steps:

1) Identify input size → n
2) Count number of operations
3) Ignore constants
4) Take highest order term

Time Complexity

Example 1 – C code:

```
for (int i = 0; i < n; i++)
{
    printf("Hi");
}
```

- If n = 5 → how many times? → 5
- If n = 100 → how many times? → 100

The running time depends on **n**.

The time increases as input increases.

Time Complexity

Example 2 – C code:

```
printf("Hi");
```

- If n = 10 or 1000 → how many times? → 1

Time does NOT depend on input size.

Space Complexity

- **Space Complexity is the measure of the amount of memory an algorithm uses during execution as a function of the input size (n).**

- It includes:
  - ✓ Input storage
  - ✓ Auxiliary (extra) memory used by algorithm

Space Complexity

Space complexity mainly consists of **two parts**:

1. **Fixed Part**

Memory that **does not depend on input size**.

Examples:

- Variables
- Constants
- Program instructions
- Fixed-size arrays

Space Complexity

1. Fixed Part

Example:

```
a = 10
b = 20
c = a + b
```

Here memory does not change with input size.

Complexity:

O(1) → Constant space

Space Complexity

2. Variable Part

Memory that **depends on the input size (n)**.

Examples:

- Dynamic arrays
- Recursion stack
- Linked list nodes
- Input data

Example:

```
arr = [1,2,3,4,5]
```

If the array size increases, memory required also increases.

Space Complexity

Formula for Space Complexity

Total Space Complexity:

$$S(n) = Fixed\ Space + Variable\ Space$$

Where

- **Fixed Space** → constant memory
- **Variable Space** → memory depending on input size n

Space Complexity

Example 1 – C code:

```
int a = 10;
int b = 20;
```

Memory used? → Fixed

Does not change with input.

Space Complexity

Example 2 – C code:

```
int arr[n];
```

If `n` increases → memory increases.

Memory usage depends on input size.

Space Complexity

Example 3 – Constant Space

```
def sum(a, b):
    c = a + b
    return c
```

Only **one extra variable (c)** is used.

Space Complexity = **O(1)**

Space Complexity

Example 4 – Linear Space

```
def create_array(n):
    arr = []
    for i in range(n):
        arr.append(i)
    return arr
```

Array size grows with **n**.
Space Complexity = **O(n)**

Space Complexity

Recursion uses **extra memory for the call stack**.

Example 5

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)
```

Each recursive call uses stack memory.
Space Complexity = **O(n)**

Space Complexity

Common Space Complexities

Space Complexity	*Meaning*	*Example*
O(1)	Constant space	Swapping numbers
O(log n)	Logarithmic space	Binary search recursion
O(n)	Linear space	Arrays, storing n elements
$O(n^2)$	Quadratic space	2D matrices

AMITY UNIVERSITY BENGALURU

Module 1 – Algorithm Analysis and Basic Concepts

- Introduction to Data Structures and Algorithms
- Abstract Data Type (ADT)
- Need for efficient algorithms
- Time Complexity and Space Complexity
- **Asymptotic Notations: Big-O, Big-Ω, Big-Θ**
- Best, Average and Worst case analysis
- Growth rate of functions (constant, logarithmic, linear, quadratic)

Asymptotic Notations

Asymptotic analysis studies performance when:

n → very large

n – input size

It ignores:

- Small inputs
- Constants
- Machine differences

Asymptotic Notations

Asymptotic Notations are mathematical tools used to describe the growth rate of an algorithm's time or space complexity as input size (n) becomes very large.

- They tell us how fast an algorithm grows when input becomes large.
- We do **not** measure exact time.
- We measure **growth behavior**.

Asymptotic Notations

Why do we need Asymptotic Notations?

- Different computers run at different speeds.
- We ignore small constants.
- We focus on large input sizes.

For example:

Algorithm A → 5n + 2

Algorithm B → (n^6) + 3

For large **n**, which grows faster?

Clearly, **(n^6) grows much faster than n**.

So, we use asymptotic notation to compare them.

Asymptotic Notations

There are **three Types of Asymptotic notations**:

1) Big-O (O)

2) Big-Omega (Ω)

3) Big-Theta (Θ)

Asymptotic Notations

(1) Big-O Notation (O)

Big-O describes the **upper bound** (worst-case growth) of an algorithm.

It tells us: "**Maximum time** an algorithm can take."

Asymptotic Notations

(1) Big-O Notation (O)

Example – C code:

```
for (int i = 0; i < n; i++)
    printf("Hi");
```

Runs **n** times.
So, we say:
Time Complexity = **O(n)**

The algorithm will not take more than proportional to **n** time.

Asymptotic Notations

(2) Big-Omega Notation (Ω)

Big-Omega describes the **lower bound** (best-case growth) of an algorithm.

It tells us: "**Minimum time** the algorithm will take."

Example: Linear Search

If element is found at first position:

Only 1 comparison.

Best case = **Ω(1)**

Asymptotic Notations

(3) Big-Theta Notation (Θ)

Big-Theta describes the **exact bound** (both upper and lower bound).

It tells us: "**Actual growth** rate of the algorithm."

- **Big-Theta (Θ)** notation represents the **tight bound** or **exact growth rate** of an algorithm.
- It means the running time grows **both at least and at most at the same rate** for large input size *n*.

Asymptotic Notations

(3) Big-Theta Notation (Θ)

Example – C code:

```
for (int i = 0; i < n; i++)
    printf("Hi");
```

- The loop runs **n times**.
- Every time it prints **"Hi"**.
- The number of steps is **always n**.

Case Analysis

- Best case → **n** times
- Worst case → **n** times

Since, the algorithm **always runs n times**, the exact growth is: **Θ(n)**

Asymptotic Notations

Comparison Table

Notation	***Meaning***	***Case***
O(n)	Upper bound	Worst case
Ω(n)	Lower bound	Best case
Θ(n)	Exact bound	Average / Tight bound

Module 1 – Algorithm Analysis and Basic Concepts

- Introduction to Data Structures and Algorithms
- Abstract Data Type (ADT)
- Need for efficient algorithms
- Time Complexity and Space Complexity
- Asymptotic Notations: Big-O, Big-Ω, Big-Θ, Best
- **Best, Average and Worst Case analysis**
- Growth rate of functions (constant, logarithmic, linear, quadratic)

Best, Average and Worst Case analysis

1. Best Case

Best case represents the **minimum time** taken by an algorithm to complete the task.

- It shows the **fastest possible execution** of an algorithm.
- It happens when the **desired element is found immediately**.
- The number of operations is **minimum**.

Best, Average and Worst Case analysis

1. Best Case

Example – Linear Search:

Given List of names:

[Deeraj, Manu, Akmal, Aditya, Sachin, Balaji, Pavan]

Suppose we want to search for a name in the list.

Best, Average and Worst Case analysis

1. Best Case

```
[Deeraj, Manu, Akmal, Aditya, Sachin, Balaji, Pavan]
```

Example – Linear Search:

Search for **Deeraj**

The algorithm checks the **first element** and finds it immediately.

Comparisons = **1**

Time Complexity: O(1)

Best, Average and Worst Case analysis

2. Average Case

Average case represents the **expected time taken** by an algorithm for a typical input.

- It shows the **average performance** of the algorithm.
- The element may appear **anywhere in the list**.
- On average, the algorithm checks **half of the elements**.

Best, Average and Worst Case analysis

```
[Deeraj, Manu, Akmal, Aditya, Sachin, Balaji, Pavan]
```

2. Average Case

Example – Linear Search:

Search for **Aditya**

Search process:

- Deeraj
- Manu
- Akmal
- Aditya ✓

Comparisons ≈ **n / 2**

Time Complexity: O(n)

Best, Average and Worst Case analysis

3. Worst Case

Worst case represents the **maximum time** taken by an algorithm.

- It shows the **slowest execution** of the algorithm.
- The element is **at the last position** or **not present in the list**.
- The algorithm must check **all elements**.

Best, Average and Worst Case analysis

3. Worst Case

Example – Linear Search:

Search for **Pavan**

Search process:

- Deeraj
- Manu
- Akmal
- Aditya
- Sachin
- Balaji
- Pavan ✓

Comparisons = **n**

Time Complexity: O(n)

```
[Deeraj, Manu, Akmal, Aditya, Sachin, Balaji, Pavan]
```

Best, Average and Worst Case analysis

Case	*Situation*	*Comparisons*	*Complexity*
Best Case	Element at first position	**1**	**O(1)**
Average Case	Element in middle	**n/2**	**O(n)**
Worst Case	Element at last / **not present**	**n**	**O(n)**

Best, Average and Worst Case analysis

- For **Best, Average, and Worst Cases**,

  Big-O is the most commonly used notation to describe time complexity.

Case	*Comparisons*	*Complexity*
Best Case	1	O(1)
Average Case	n/2	O(n)
Worst Case	n	O(n)

Best, Average and Worst Case analysis

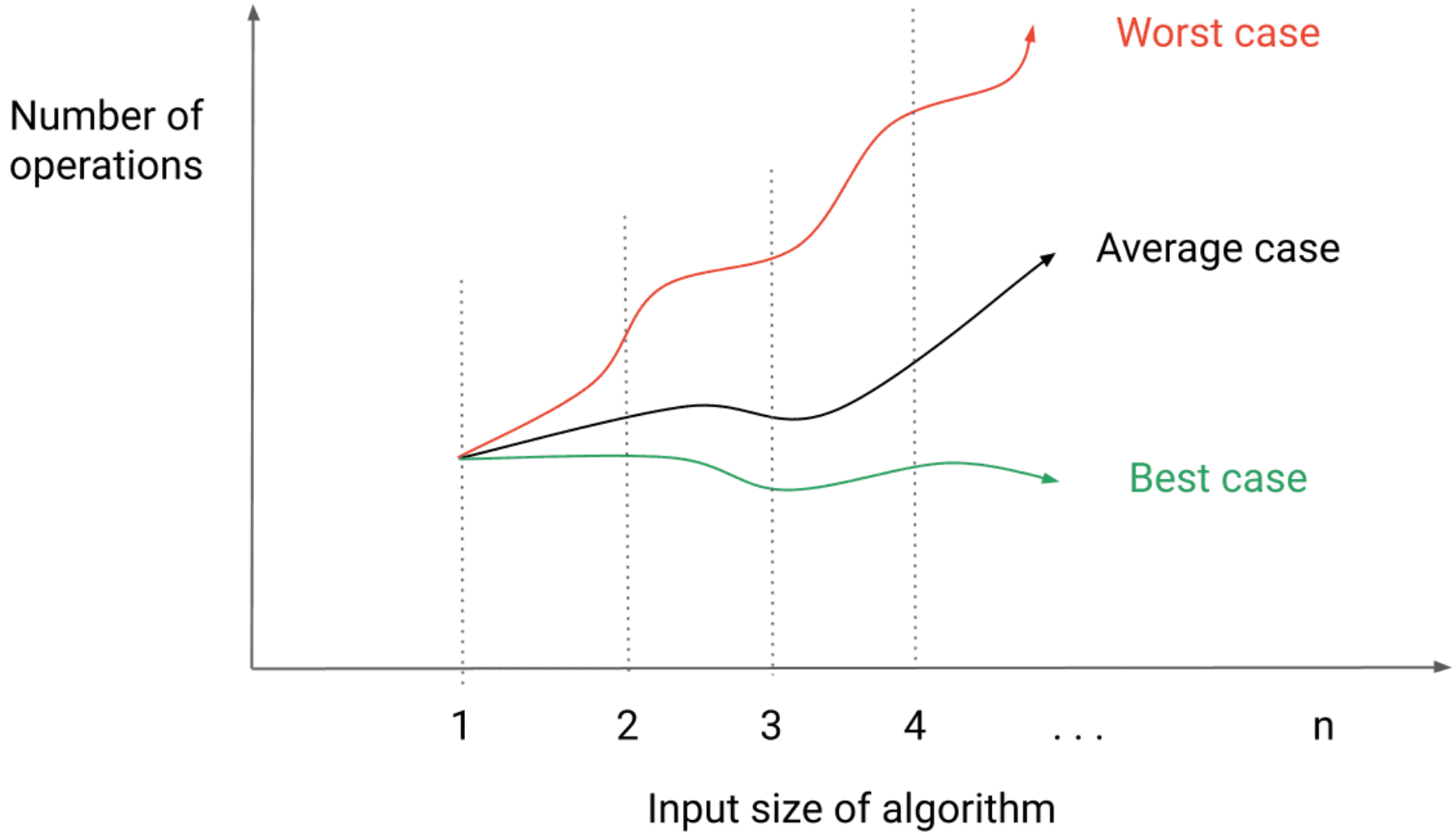
- If we use the **asymptotic notations**, it would be:

Case	*Notation*	*Meaning*
Best Case	Ω(1)	Minimum time
Average Case	Θ(n)	Typical growth
Worst Case	O(n)	Maximum time

Example: **Linear Search**

- Best case → element at first position → **Ω(1)**
- Average case → element somewhere in list → **Θ(n)**
- Worst case → element at last / not found → **O(n)**

Best, Average and Worst Case analysis



- Best Case → **Fastest**
- Average Case → **Normal performance**
- Worst Case → **Slowest**

AMITY UNIVERSITY BENGALURU

Module 1 – Algorithm Analysis and Basic Concepts

- Introduction to Data Structures and Algorithms
- Abstract Data Type (ADT)
- Need for efficient algorithms
- Time Complexity and Space Complexity
- Asymptotic Notations: Big-O, Big-Ω, Big-Θ
- Best, Average and Worst case analysis
- **Growth rate of functions (constant, logarithmic, linear, quadratic)**

Growth rate of functions (constant, logarithmic, linear, quadratic)

- **Growth rate of functions** describes how the running time (or number of operations) of an algorithm increases as the **input size (n)** increases.
- This is to compare algorithms and is usually expressed using **Big-O notation**.

Common growth rates:

1) Constant Growth – **O(1)**
2) Logarithmic Growth – **O(log n)**
3) Linear Growth – **O(n)**
4) Quadratic Growth – **O(n^2)**

Growth rate of functions (constant, logarithmic, linear, quadratic)

1) Constant Growth – O(1)

$$f(x) = c$$

c represents a **constant value**

- The running time **does not depend on the input size**.
- No matter whether the input size is 10, 1000, or 1,000,000, the algorithm takes **the same amount of time**.

Example

Accessing an element in an array by index.

```
arr = [10, 20, 30, 40, 50]
print(arr[2])    # Access element
```

Growth rate of functions (constant, logarithmic, linear, quadratic)

1) Constant Growth – O(1)

Operations

Only **one step** is needed to access the element.

Real-world analogy

Picking a specific book from a shelf **when you already know the exact position**.

Growth rate of functions (constant, logarithmic, linear, quadratic)

2) Logarithmic Growth – O(log n)

$$f(x) = \log x$$

- The running time **increases slowly** as input grows.
- At every step, the problem size is **reduced by half**.

Example

Binary Search

Instead of checking every element, we repeatedly divide the list.

Example list (sorted):

```
[5, 10, 15, 20, 25, 30, 35]
```

Growth rate of functions (constant, logarithmic, linear, quadratic)

2) Logarithmic Growth – O(log n)

Example list (sorted): `[5, 10, 15, 20, 25, 30, 35]`

Steps to find **25**:

1) Check middle → 20
2) Go right half
3) Check middle → 30
4) Go left → 25

Only **few steps**, even for very large lists.

Real-world analogy

Looking for a word in a **dictionary** by opening the middle page.

Growth rate of functions (constant, logarithmic, linear, quadratic)

3) Linear Growth – O(n)

$$f(x) = x$$

- The running time **increases directly with input size**.
- If input doubles → time also doubles.

Example

Searching an element in an unsorted list.

```
arr = [10, 25, 30, 45, 50]

for x in arr:
    if x == 45:
        print("Found")
```

Growth rate of functions (constant, logarithmic, linear, quadratic)

3) Linear Growth – O(n)

Steps:

Input Size	*Steps*
5 elements	up to 5 checks
100 elements	up to 100 checks
1000 elements	up to 1000 checks

Real-world analogy

Checking **every student name in a list** until you find the correct one.

Growth rate of functions (constant, logarithmic, linear, quadratic)

4) Quadratic Growth – $O(n^2)$

$$f(x) = x^2$$

- The output increases **proportional to the square of the input**.
- If the input doubles, the output becomes **four times larger**.

Example **(Nested loops)**

```
for i in range(n):
    for j in range(n):
        print(i, j)
```

- The outer loop runs **n times**.
- The inner loop also runs **n times** for each iteration.
- Total operations = **$n \times n = n^2$**

Growth rate of functions (constant, logarithmic, linear, quadratic)

4) Quadratic Growth – $O(n^2)$

$$f(x) = x^2$$

Example Values

x	$f(x) = x^2$
1	1
2	4
3	9
5	25
10	100

This shows the **rapid increase in output** as x grows.

Growth rate of functions (constant, logarithmic, linear, quadratic)

4) Quadratic Growth – $O(n^2)$

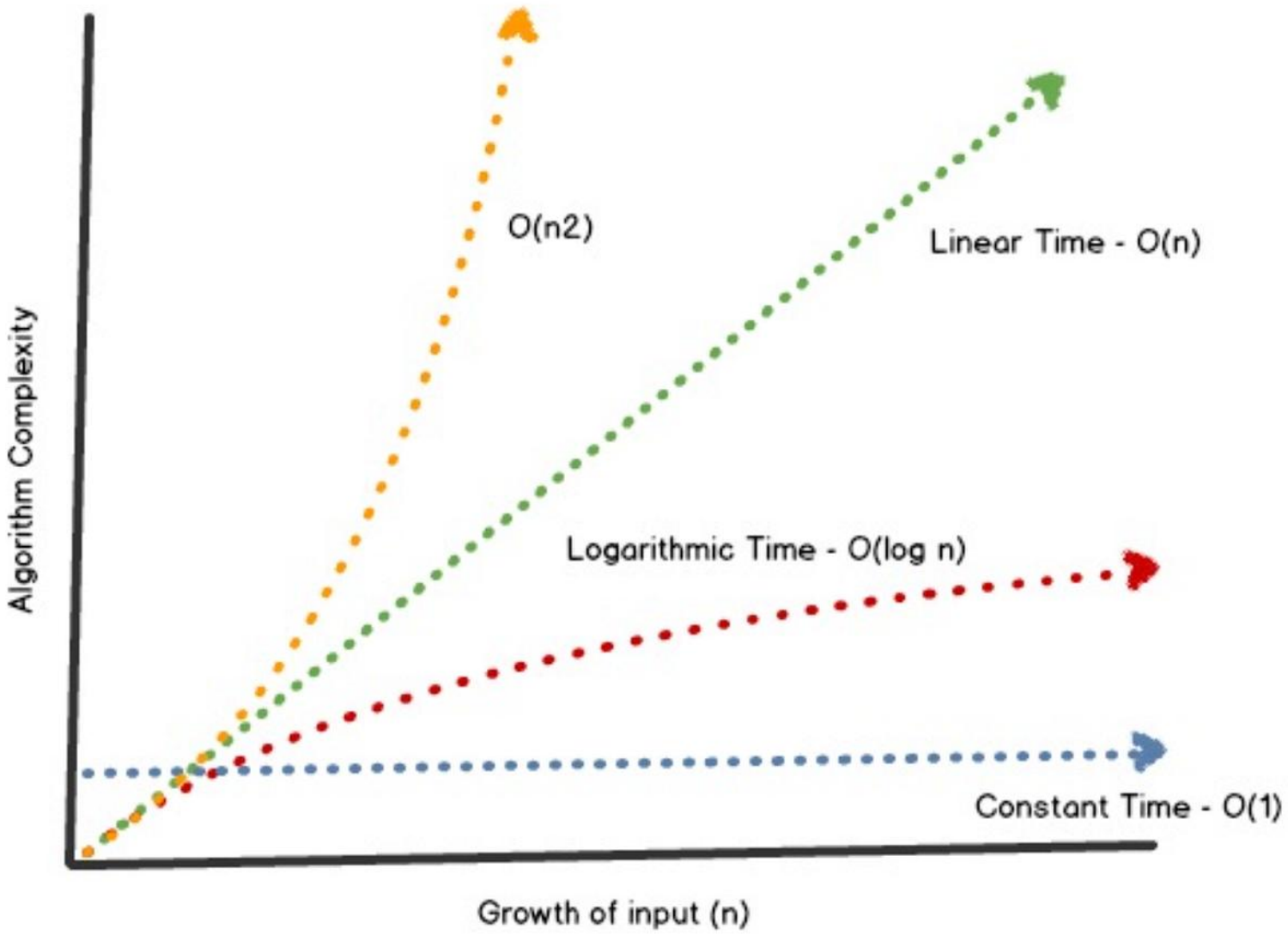
Steps:

Input Size	*Operations*
n = 10	100
n = 100	10,000
n = 1000	1,000,000

Real-world analogy

Comparing **every student with every other student** in a class.

Growth rate of functions (constant, logarithmic, linear, quadratic)



Growth rate of functions (constant, logarithmic, linear, quadratic)

Comparison of Growth Rates

Growth Type	*Big-O*	*Speed*
Constant	O(1)	Fastest
Logarithmic	O(log n)	Very efficient
Linear	O(n)	Moderate
Quadratic	$O(n^2)$	Slow for large data

As the input size **n increases**, algorithms with **smaller growth rates perform better**.

Order or Ranking (best → worst):

$$O(1) < O(\log n) < O(n) < O(n^2)$$

Growth rate of functions (constant, logarithmic, linear, quadratic)

Comparison of Growth Rates

Growth Type	Big-O Notation	Mathematical Form	Description
Constant	O(1)	f(x) = c	Running time does not change with input size
Logarithmic	O(log n)	f(x)= log x	Running time increases slowly as input grows
Linear	O(n)	f(x) = x	Running time increases directly with input size
Quadratic	$O(n^2)$	f(x) = x^2	Running time grows rapidly due to nested loops

Practice Problems

Practice Problems

1. **Identify the Time Complexity**

Find the **time complexity** of the following code:

```
for i in range(n):
    print(i)
```

Question:

What is the time complexity?

A. O(1)

B. O(log n)

C. O(n)

D. $O(n^2)$

Practice Problems

2. Nested Loop Complexity

```
for i in range(n):
    for j in range(n):
        print(i, j)
```

Question:

What is the time complexity?

A. O(n)

B. O(n log n)

C. $O(n^2)$

D. $O(n^3)$

Practice Problems

3. Logarithmic Growth

```
i = 1
while i < n:
    i = i * 2
```

Question:

What is the time complexity?

A. O(n)

B. O(log n)

C. $O(n^2)$

D. O(1)

Practice Problems

4. Space Complexity

```
def sum_numbers(n):
    total = 0
    for i in range(n):
        total += i
    return total
```

Question:

What is the **space complexity**?

A. O(1)

B. O(n)

C. O(n^2)

D. O(log n)

Practice Problems

5. Recursion Space Complexity

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n-1)
```

Question:
What is the **space complexity**?

Practice Problems

6. Best, Average, Worst Case

For **Linear Search** in an array:

```
for i in range(n):
    if arr[i] == key:
        return I
```

Questions

1.Best Case Time Complexity = ?

2.Average Case Time Complexity = ?

3.Worst Case Time Complexity = ?

Practice Problems

7. Growth Rate Ordering

Arrange the following functions from **slowest growth to fastest growth**.

1) $\log n$
2) n^2
3) n
4) 1

Practice Problems

8. Identify the Growth Rate

Which growth rate represents **constant time**?

A. $f(n) = 1$

f(n)=1

B. $f(n) = \log n$

f(n)=log n

C. $f(n) = n$

f(n)=n

D. $f(n) = n^2$

f(n)=n^2

Practice Problems

9. Asymptotic Notation

Match the notation with its meaning.

Notation	*Meaning*
Big-O	?
Big-Ω	?
Big-Θ	?

Options:

A. Upper bound

B. Lower bound

C. Tight bound

Practice Problems

10. Identify the Complexity

```
for i in range(n):
    for j in range(i):
        print(i, j)
```

Question:
What is the time complexity?

Practice Problems

Answer Key

Problem	Answer
1	O(n)
2	$O(n^2)$
3	O(log n)
4	O(1)
5	O(n)
6	Best: O(1), Average: O(n), Worst: O(n)
7	1 → log n → n → n^2
8	f(n) = 1
9	Big-O = Upper bound, Big-Ω = Lower bound, Big-Θ = Tight bound
10	$O(n^2)$

THANK YOU

Dr. Vignesh Ramamoorthy. H

Associate Professor

AIIT – AUB